High-Availability Layer 7 Load Balancer — Report

Sai Karthikeyan Sura

April 19, 2026

https://github.com/karthikeyansura/ha-l7-lb

Extended methodology, raw data paths, related work, and appendix: /docs/final_report.md

1 Introduction and system overview

Production load balancers hide operational trade-offs: cost of stateful routing vs round-robin, when retries help or hurt, and when coordination limits horizontal scale. This work implements a custom Layer 7 reverse proxy in Go—round-robin, least-connections (power-of-two choices), weighted routing; active health checks; idempotent retries with a 20% budget; Redis Pub/Sub for cross-LB health; Cloud Map DNS discovery—deployed on AWS ECS Fargate behind an NLB. Three experiments quantify routing on homogeneous and heterogeneous backends, retry behavior under chaos vs a single replica loss, and scaling the LB tier for an LB-bound vs backend-bound workload. Headline findings (detailed below): weighted routing holds a 70/30 split within 0.5 pp under two workloads; retries yield ∼44× fewer failures under narrow failure but ∼10× *more* failures under broad chaos; LB scaling raises aggregate throughput on light `/api/data` but not on CPU-bound `/api/compute`.

2 Methodology

Client → NLB (TCP) → LB ECS tasks → backends; ElastiCache Redis synchronizes health; Cloud Map DNS discovers backends (multi-source pools for strong/weak tiers). Locust drives the NLB from a same-VPC EC2 host. Chaos uses `X-Chaos-Error` and `X-Chaos-Delay` on `/api/data` and `/api/compute`. The LB exposes `/metrics` (and related endpoints) on port + 1000. Full setup, infrastructure layout, and AI-disclosure appear in /docs/final_report.md

3 Experiment 1: Routing algorithm comparison

Purpose, trade-off, and limitations

Purpose: Measure whether RR, LC, and Weighted change throughput, tail latency, and per-backend share on identical vs unequal backends, including a mixed compute/payload/stream workload.
Trade-off: Stateless fairness (RR) vs local load awareness (LC) vs operator-specified capacity split (Weighted).
Limitations: Heterogeneous topology is one strong and one weak task; LB can be CPU-saturated at 500 users, masking algorithm differences on `/api/data`; weighted counts requests, not CPU or bytes; single-run cells in the robustness matrix.

Results

Milestone `/api/data` (homogeneous). LC and Weighted reduce tail latency vs RR substantially:

Metric	RR	LC	Weighted
RPS	1,269	1,482	1,523
p99	920 ms	440 ms	410 ms
Max	2,698 ms	1,314 ms	663 ms

Weighted 70/30 (heterogeneous), from LB `metrics` aggregates: strong 70.0%, weak 30.0% (500 users; RPS ≈2,406, p99 180 ms). Figure 1 shows the CloudWatch dashboard; companion exp1-dashboard_end1.png matches the second LB task.

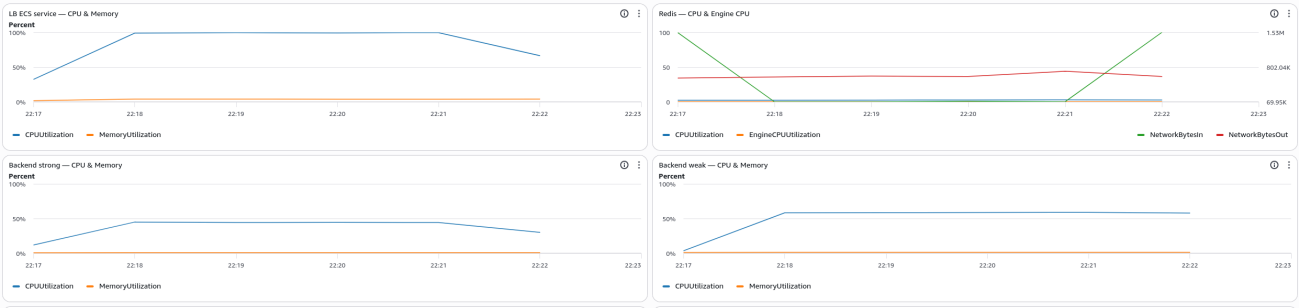

Figure 1: Weighted 70/30 hetero run: LB at ∼100% CPU; strong ∼45%, weak ∼55% despite 30% traffic share—weak has half the CPU budget.

Robustness matrix (`BackendStressUser`, 20 users, 5 min): all six cells zero failures; hetero distribution (LB view):

Topo	Policy	RPS	p99	Strong/Weak %
Homo	RR	52.6	2.1 s	—
Homo	LC	53.2	2.2 s	45.1 / 54.9
Homo	W(100)	49.9	2.4 s	49.9 / 50.1
Hetero	RR	52.0	2.3 s	50.1 / 49.9
Hetero	LC	54.6	2.2 s	53.4 / 46.6
Hetero	W(70/30)	53.8	2.2 s	**69.8 / 30.2**

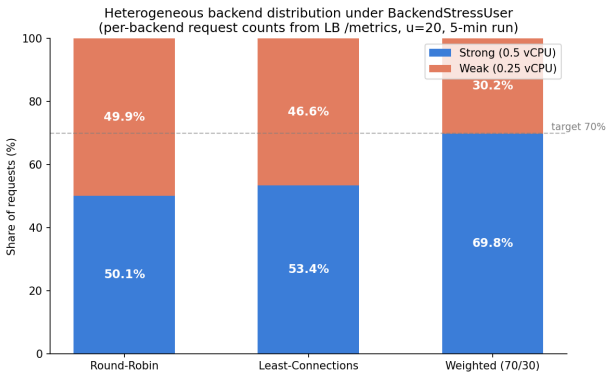

Figure 2: Heterogeneous backends: RR near 50/50, LC mild bias, Weighted matches 70/30 within 0.5 pp.

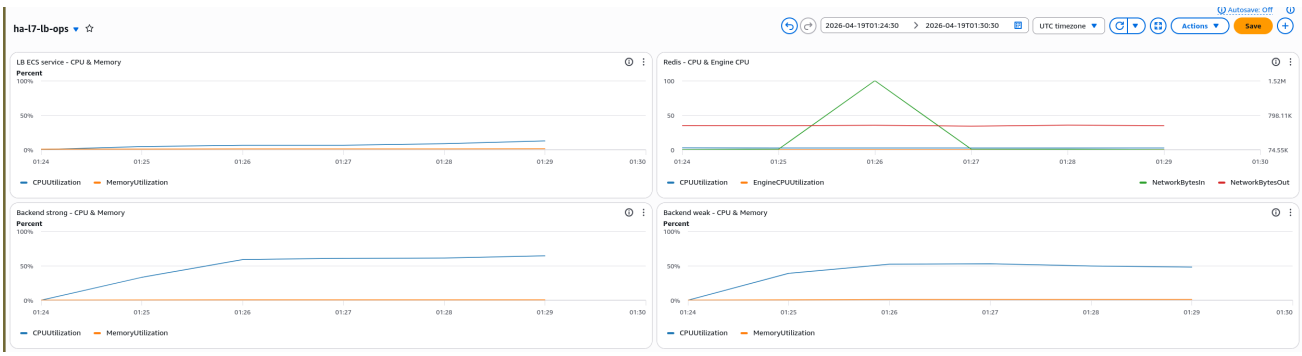

Figure 3: Hetero Weighted robustness: strong ∼65% CPU, weak ∼50%—aligned with configured split and 2:1 capacity.

Analysis

On identical backends RR, LC, and Weighted converge in distribution but LC/Weighted still win on `/api/data` tails (scheduling variance). On hetero backends the strong-share progression 50→53→70 shows RR blind, LC partial, Weighted exact. Mixed-workload p99∼2 s is dominated by the 2 s streaming hold, not routing overhead. **Conclusion:** If capacity ratios are known, Weighted is the safe default; LC needs enough load to differentiate queues; RR overloads the weak node.

4 Experiment 2: Retry efficacy and cascading health

Purpose, trade-off, and limitations

Purpose: Measure client-visible failures with retries on vs off under controlled chaos and under a single mid-run replica drop.
Trade-off: Fast fail-over via retry+DOWN-mark vs stability under correlated failures.
Limitations: Chaos is synthetic and header-forwarded on retry; original 30% chaos collapses both arms (see /results/final/exp2/); low-chaos (∼10%) chosen to expose deltas; CPU chaos variant adds a second workload shape.

Results

Part A, low chaos on `/api/data`:

Users	Retries	Total req	Failures	Fail %	p99
50	on	45,529	41,621	91.4%	23 ms
100	on	95,590	93,145	97.4%	11 ms
200	on	192,943	190,665	98.8%	2 ms
50	off	26,456	2,517	9.5%	5000 ms
100	off	53,714	4,985	9.3%	5000 ms
200	off	106,590	10,100	9.5%	5000 ms

Part A, compute chaos (200 users): retry on 98.5% failures vs retry off 9.4%. Figure 4 contrasts backend CPU: ∼0% during retry-on cascades vs ∼65% when retries are off and legitimate traffic proceeds.

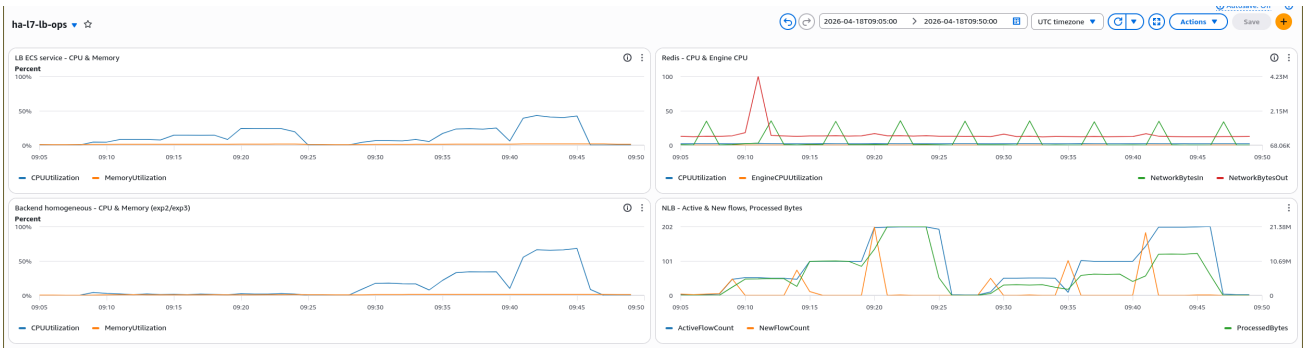

Figure 4: Six compute-chaos runs: backend CPU flat under retry-on cascades; active under retry-off.

Part B, replica drop at T+150 s (200 users):

Variant	Requests	Failures	Fail %	RPS	p99
retry_on	850,924	18	0.002%	1,421	180 ms
retry_off	692,695	784	0.113%	1,157	270 ms

LB snapshot (retry-on path): 4 retries / ∼123k requests on lb0; 50% retry success (2/4). Figure 5.

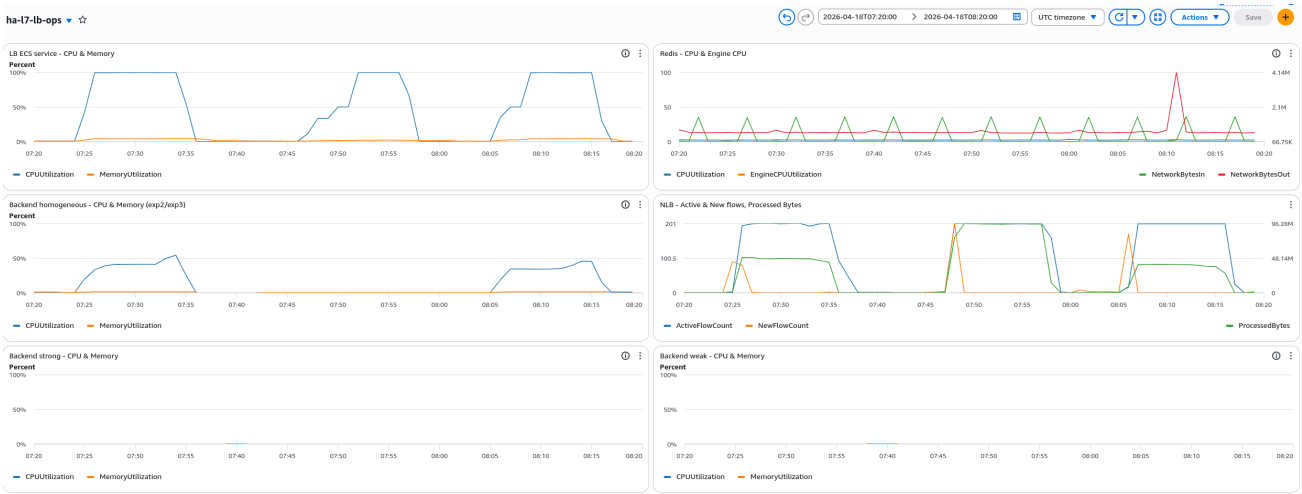

Figure 5: Part B CloudWatch: LB CPU plateaus for retry-on vs retry-off; Redis egress spike on retry-off marks DOWN propagation after replica drop.

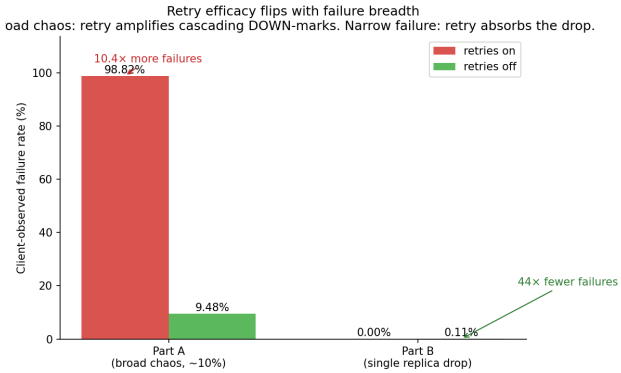

Figure 6: Failure-rate flip: broad chaos favors retry-off; narrow replica loss favors retry-on (≈44× fewer raw failures).

Analysis

Under broad chaos, retries spread 5xx-induced DOWN states via Redis until the pool empties (fast errors, near-zero p99). Under a single backend loss, sparse retries reach healthy tasks (≈44× fewer failures). **Conclusion:** Same mechanism, opposite outcome depending on failure breadth; production systems decouple retry from ejection (rate-based, consecutive failures, half-open).

5 Experiment 3: Horizontal LB scaling

Purpose, trade-off, and limitations

Purpose: Measure aggregate throughput and tails as LB count scales 1–8 for an LB-light workload (`/api/data`) vs backend-heavy `/api/compute`.
Trade-off: Throughput vs coordination cost (Redis); autoscaling the correct tier.
Limitations: Single region; Redis not saturated at these scales; burst vs steady difference <1% RPS in reported spike runs; Exp 3b shows overload can break health probes (shared CPU with `/health`)—documented negative result.

Results

lb	u=500 RPS	u=2000 RPS	u=2000 p99
1	2,427	1,188	11,000 ms
2	2,794	2,699	10,000 ms
4	4,308	4,757	7,200 ms
8	5,777	5,093	250 ms

Compute scaling flat ∼55 RPS for lb 1 . . . 8; p99 ∼2.1 s. Figures 7–9.

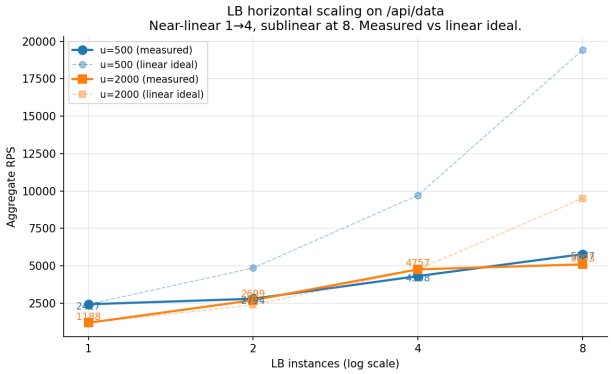

Figure 7: RPS vs LB count for `/api/data`; dashed linear ideal from lb=1.

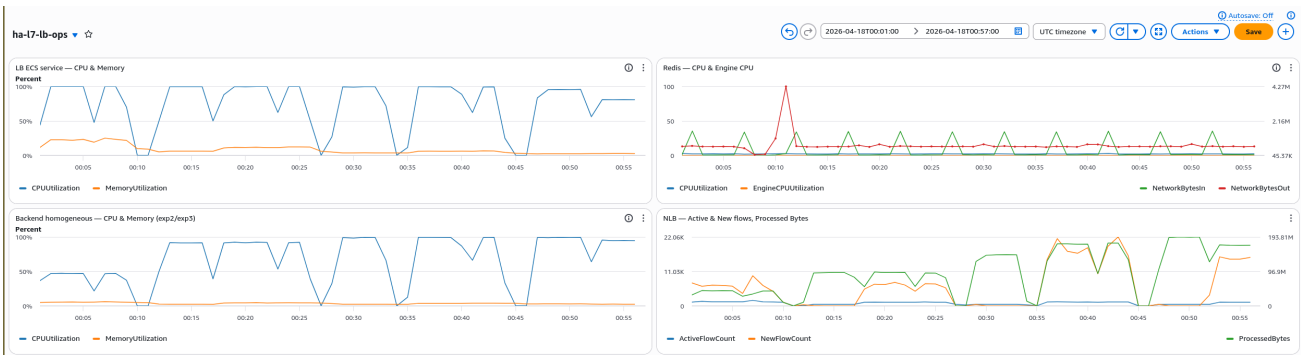

Figure 8: u=2000 sweep: LB and backend CPU saturate; Redis CPU remains ∼0—coordination not the limiter.

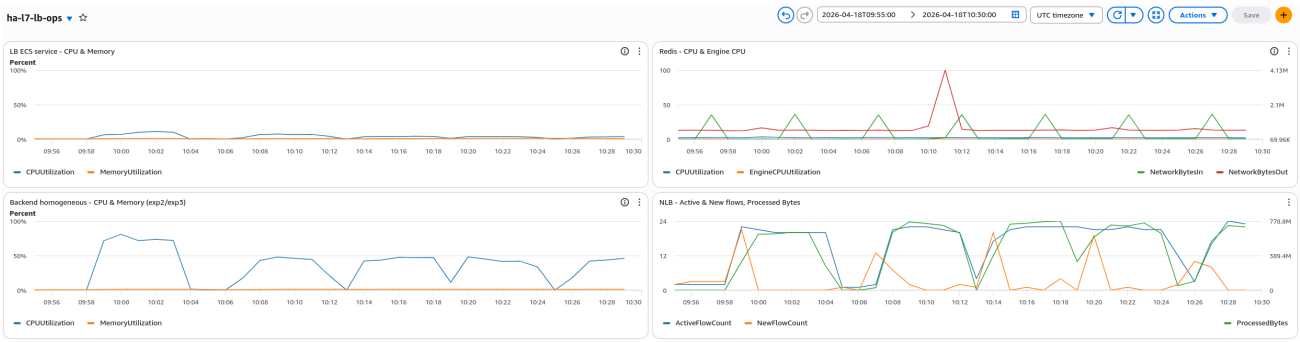

Figure 9: Compute scaling runs: LB CPU <15%; backend CPU is the pacing signal—extra LB tasks do not add throughput.

Exp 3b (overload): /results/final/exp3b/README.md—∼95–99% failures from health-timeouts under CPU overload, even with retries disabled.

Analysis

On `/api/data`, aggregate RPS grows with LB count until backends or LB saturation patterns shift; at u=2000, scaling to lb=8 cuts p99 from ∼11 s to 250 ms (∼44×). On `/api/compute`, backends cap throughput—scaling LBs wastes resources. Exp 3b highlights health-endpoint isolation. **Conclusion:** Autoscale the bottlenecked tier; verify health paths under load.

6 Discussion, conclusions, limitations

Cross-experiment takeaway: combine Locust CSVs, CloudWatch, and LB `/metrics`—any one view can mislead. Key limitations (see /docs/final_report.md §7.2): health probes share backend CPU; aggressive per-5xx DOWN-marking; no session affinity; single region; hetero fleet size fixed to two tiers.

Data and reproduction. All figures and tables trace to /results/milestone_1/, /results/final/, and /results/figures/; see Appendix §9 of /docs/final_report.md for the file-level index.